Year: 2026-2027 **Sem V-NEP**

Course: Agile Software Development & DevOps Lab

Date of Performance: 04/08/2026 **Date of Submission: 04/08/2026**

Class: T1

Student Roll No: 2401036 **Student Name: Dhruv Hemdev**

Experiment 4

Aim: To set up a Continuous Integration Pipeline using Jenkins/GitHub Act.

This experiment maps to LO3 To set up a Continuous Integration Pipeline using Jenkins/GitHub Act.

LO3: Build and manage automated CI/CD pipelines for application, development, testing, packaging and deployment.

Theory:

CI/CD Basics

- **Continuous Integration (CI):** Automating the build and testing process every time code is changed.
- **Continuous Delivery (CD):** Keeping code in a state where it can be deployed manually at any time.
- **Continuous Deployment (CD):** Automatically deploying code to production if it passes all automated tests.

Jenkins Overview

- **What it is:** An open-source, Java-based automation server used to run CI/CD workflows.
- **Plugins:** Jenkins relies on thousands of plugins to connect with different languages, clouds, and tools.

Pipeline Architecture

- **Jenkinsfile:** A text file stored with your code that defines your automation process ("Pipeline-as-Code").
- **Agent (Node):** The physical or virtual machine where Jenkins actually executes the work.
- **Stage:** A major phase in your pipeline (e.g., Build, Test, Deploy).
- **Step:** A single command executed inside a stage (e.g., running an echo or a terminal command).

Pipeline Syntax Types

- **Declarative:** The modern, structured, and easy-to-read syntax (uses the pipeline { } block).
- **Scripted:** The older, Groovy-based syntax that offers advanced logic but is harder to learn.

Implementation:

→ Installation of Jenkins and setting up environment

1. Install and Verify Java (JDK):
Jenkins requires Java to run. Open your terminal or command prompt and verify the installed Java version

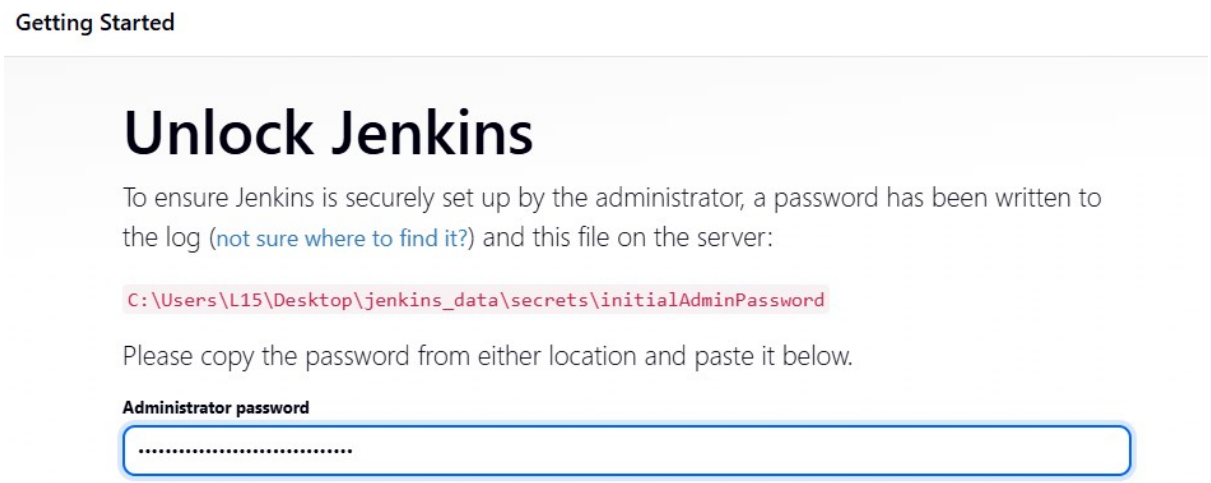
```
C:\Users\L15>java -version
java version "25.0.2" 2026-01-20 LTS
Java(TM) SE Runtime Environment (build 25.0.2+10-LTS-69)
Java HotSpot(TM) 64-Bit Server VM (build 25.0.2+10-LTS-69, mixed mode, sharing)
```

2. Download Jenkins WAR File

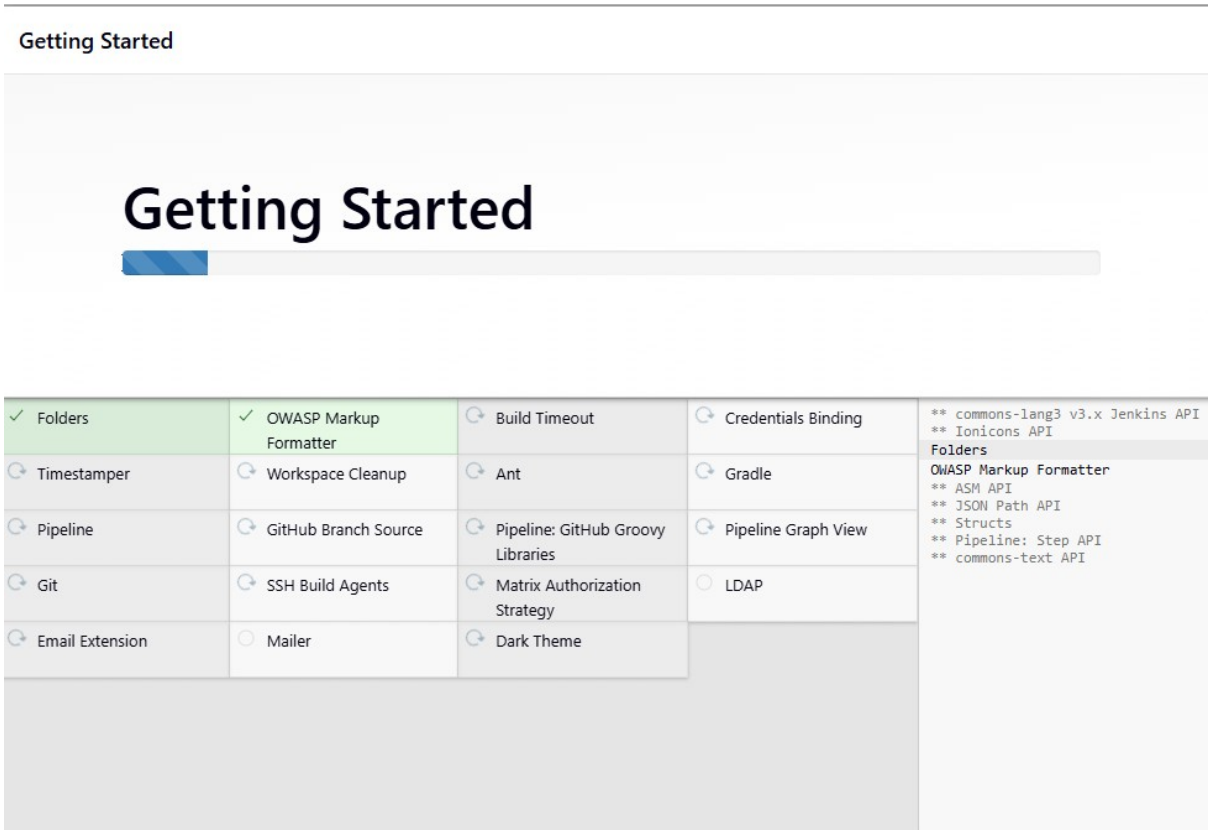
```
[CRLF]> ************************************************************
[CRLF]> ************************************************************
[CRLF]> ************************************************************
[CRLF]>
[CRLF]> Jenkins initial setup is required. An admin user has been created and a password generated.
[CRLF]> Please use the following password to proceed to installation:
[CRLF]>
[CRLF]> 42cd046347bf4e3fbb08d4323892b225
[CRLF]>
[CRLF]> This may also be found at: C:\Users\L15\Desktop\jenkins_data\secrets\initialAdminPassword
[CRLF]>
[CRLF]> ************************************************************
[CRLF]> ************************************************************
[CRLF]> ************************************************************
```

3. Launch Jenkins Server

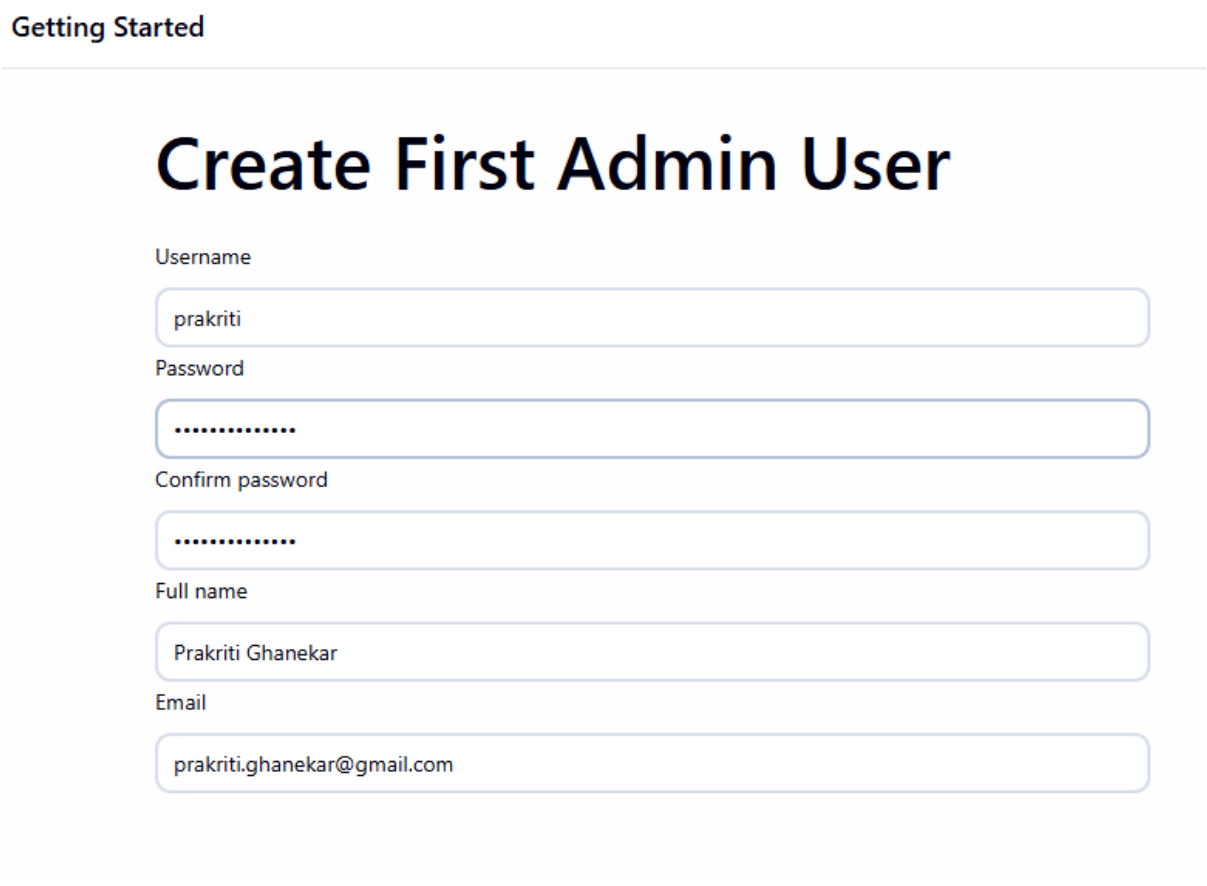
Authenticating the local Jenkins instance using the initial 32-character administrator password.



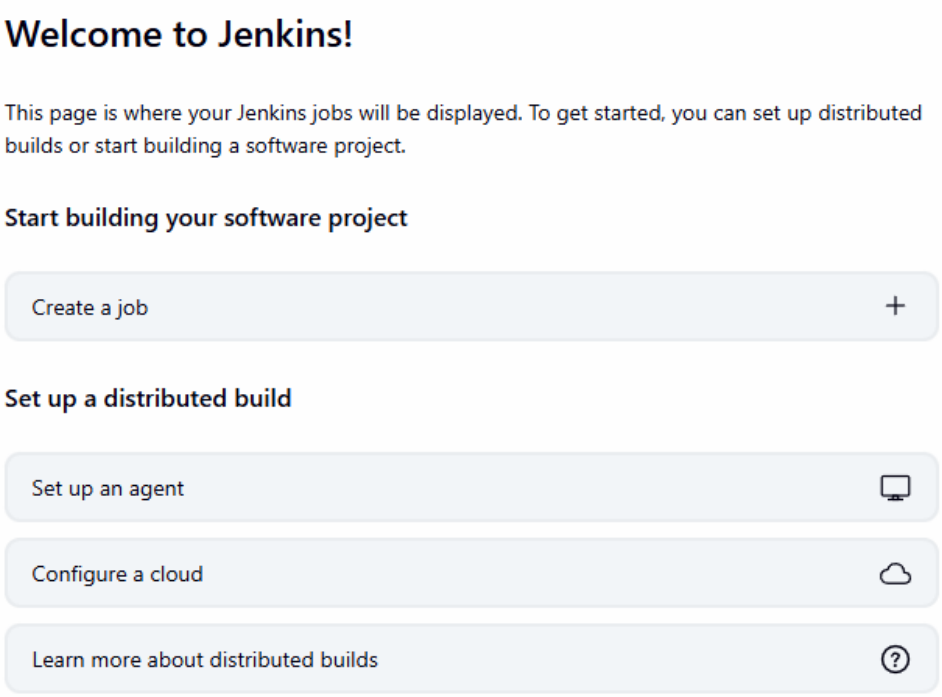
4. Install Plugins
Installing standard CI/CD plugins following successful system clock SSL certificate validation.



5. Admin User Creation
Creating the primary administrator account for workspace access.



6. Dashboard
Successful deployment and configuration of the Jenkins environment.



→ Building Jenkins Pipeline

1. Declarative Pipeline Creation

Creation of pipeline:

Pipeline script

Script ?

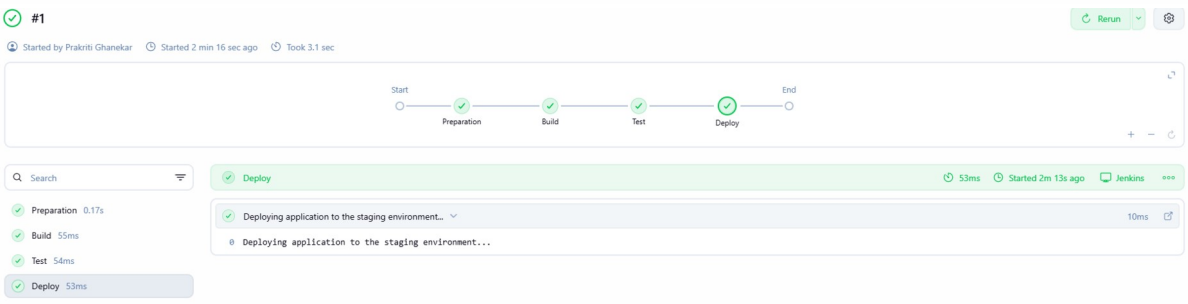
```
pipeline {
    agent any

    stages {
        stage('Preparation') {
            steps {
                echo 'Cleaning up and setting up the workspace...'
            }
        }
        stage('Build') {
            steps {
                echo 'Compiling the source code...'
            }
        }
        stage('Test') {
            steps {
                echo 'Executing unit and integration tests...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying application to the staging environment...'
            }
        }
    }
}
```

Execution of Pipeline:

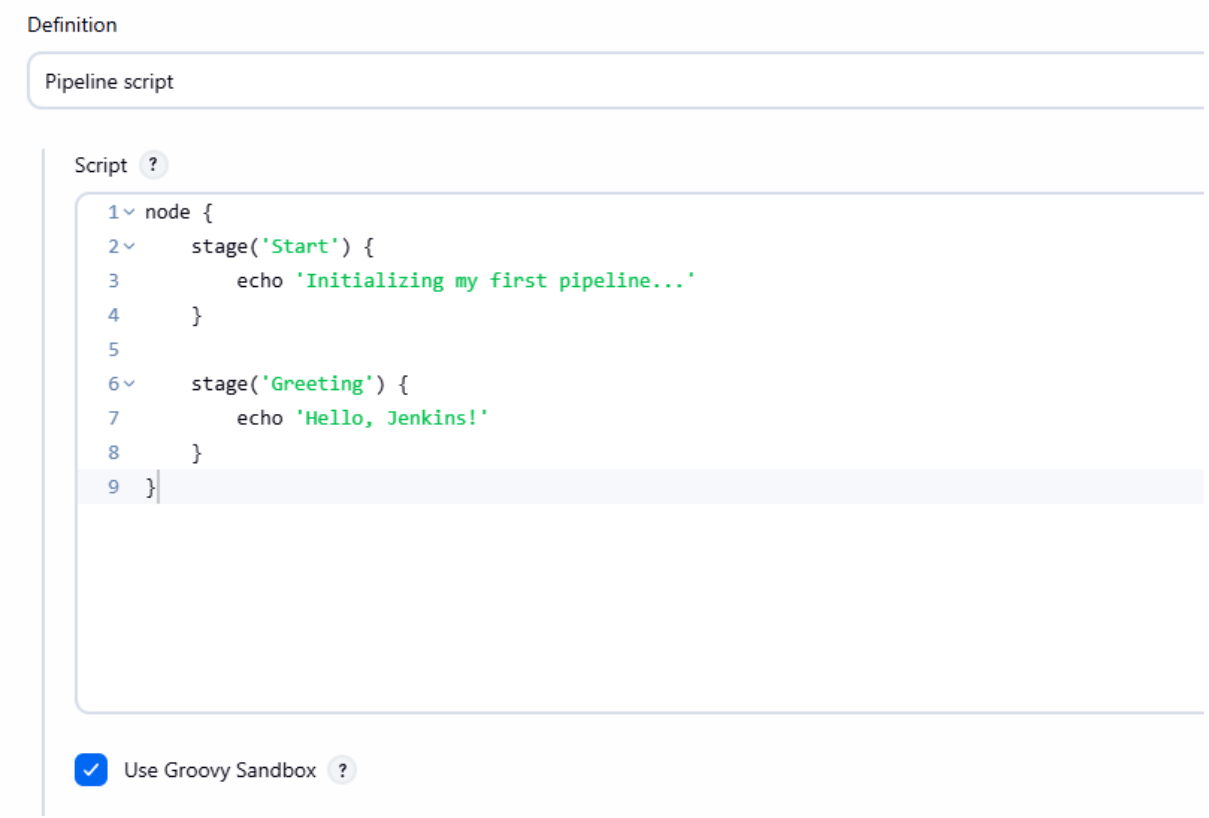
Console

```
Started by user Prakriti Ghanekar
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\Users\L15\Desktop\jenkins_data\workspace\Experiment-4-Declarative
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Preparation)
[Pipeline] echo
Cleaning up and setting up the workspace...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Compiling the source code...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Executing unit and integration tests...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying application to the staging environment...
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



2. Scripted Pipeline Creation

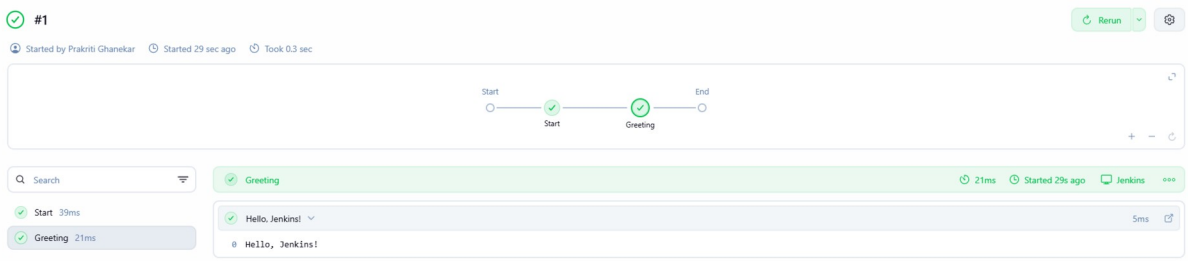
Creation of pipeline:



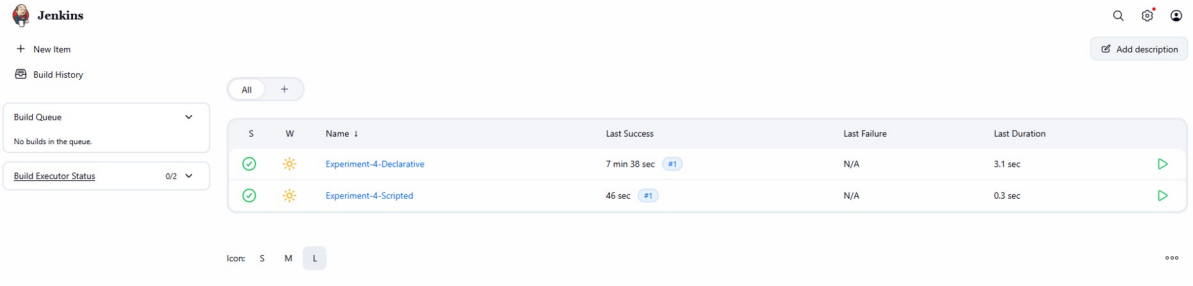
Execution of pipeline:

Console

```
Started by user Prakriti Ghanekar
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\Users\L15\Desktop\jenkins_data\workspace\Experiment-4-Scripted
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Start)
[Pipeline] echo
Initializing my first pipeline...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Greeting)
[Pipeline] echo
Hello, Jenkins!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



Pipelines on the dashboard:



Conclusion:

In this experiment, a local Jenkins automation server was successfully initialized, configured, and utilized to understand the core concepts of Continuous Integration and Continuous Delivery (CI/CD). The experiment practically explored the two primary methodologies for defining Jenkins workflows: Declarative and Scripted pipelines.

By successfully building, executing, and monitoring both pipeline types, a practical understanding was established, demonstrating how software deployment processes can be effectively automated and managed within a Jenkins environment.